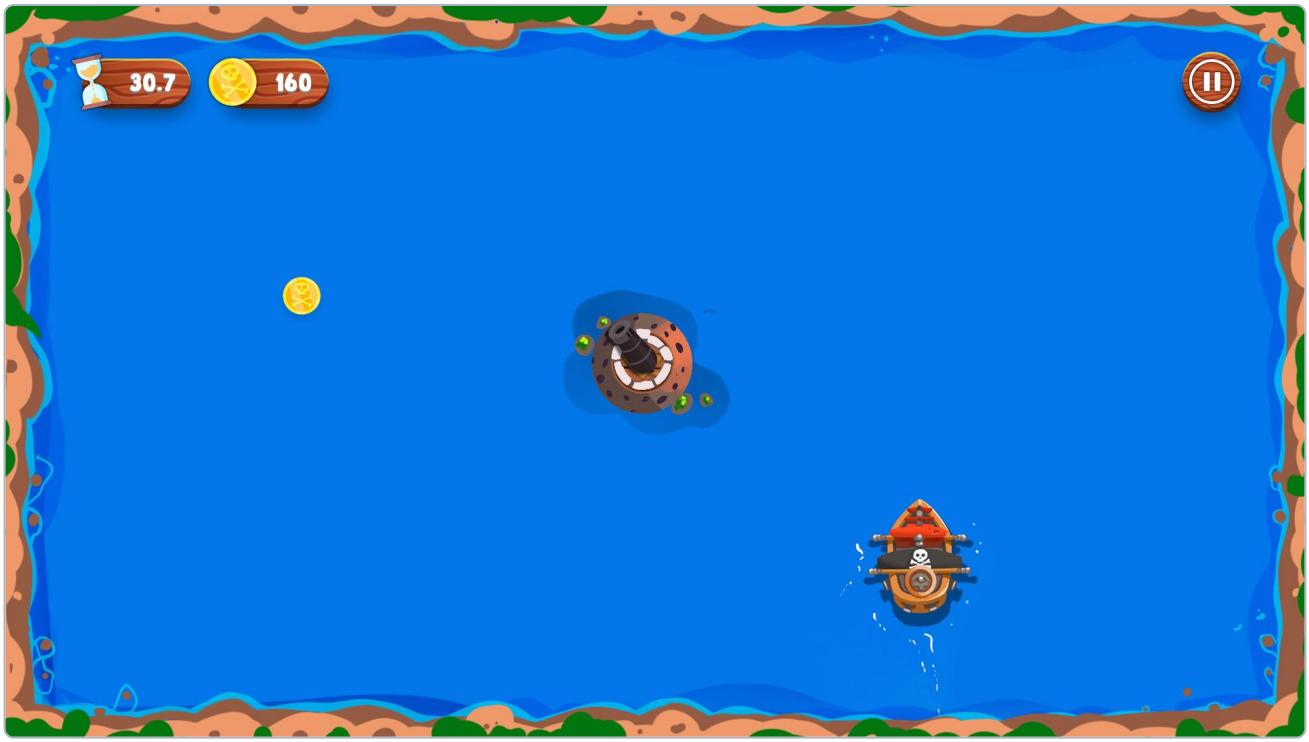
ESCAPE

Pirate survival game · Codex development brief
Prototype 1 · 21 October 2026



Original game visual reference supplied by the creator

> **Critical correction:** the rotating cannon belongs on the fortified island in the **centre of the arena**, as shown above. This overrides the location in the earliest brief.

Codex implementation specification | Prototype 1 | 21 October 2026

Status: Build instructions for a NEW repository; this is not an existing codebase and is not an instruction to recreate a different escape-room game.

1. One-paragraph concept

An arcade survival game viewed from directly above, playable in **landscape**. The player steers a small pirate ship around a bright blue ocean arena with an irregular coastal border. In the **middle** is a fixed fort island with a **continuously rotating cannon**. It fires cannonballs in changing directions; balls ricochet off the coast and persist, so the arena becomes progressively more dangerous. Avoid being hit, survive longer, gather collectible coins, and strategically pick up temporary advantages while steering clear of disadvantages. One hit from a cannonball ends the run. The core appeal is responsive movement, escalating chaos, fair collisions, and instant replayability.

2. Authority / how to interpret contradictory source documents

Read `original_briefs/` and view `assets/reference/` before implementing. The chronological source set is the 2023 initial brief, Version 2 features, Version 3 features, Version 4 bug corrections, and reference photos. **Explicit current correction (October 2026): the cannon is on the fort island in the CENTRE, as in the provided original gameplay screenshot; NOT on the edge of the arena as the earliest brief says.** Later revisions supersede earlier descriptions in other conflicts. Annotations in `annotated_design_reference.png` are visual markup; don't infer an unspoken required feature from red/green circles. Avoid exact pixel copying of those markup strokes into the game.

Required visual baseline: `assets/reference/original_gameplay_reference.jpg` shows the intended screen: 16:9 landscape, bright saturated blue water, organic orange/green cliffs along ALL four sides, central brown fortified island and cannon, top-down orange/brown pirate ship, compact wood-panel timer/coin HUD upper left, circular pause control upper right. Retain this style, overall placement, readability, and proportions as closely as practical; modernize only where needed for usability. No golf, platformer, isometric camera, or separate level progression.

The supplied `assets/source/Initial_bg_1.psd` is layered original artwork. We have also prepared **transparent approximate first-pass sprites** and a background in `assets/processed/`. Use these immediately where usable; refine extraction/masks as necessary. `arena_empty.png` contains NO baked-in moving ship or central island. **Never animate the ship or cannon by moving the entire flattened screenshot; never render a second ship or cannon on top of permanently baked-in versions.** Transparent sprites need anchored/scaled alignment and may need minor cleanup; they are starter assets, not guarantees of production-ready layer isolation.

The other photos in `original_briefs/Photo References v2.pdf` include third-party/stock imagery: **visual inspiration only**; do not ship those images as game assets without documented rights.

3. Recommended build platform

Create a **web-first Phaser 3 + TypeScript + Vite** prototype with deterministic 2D collision logic. Make it runnable with `npm install`, `npm run dev`, and `npm run build`; no Unity, Xcode, Mac-specific editor, or paid tool should be required for this first test. Mobile Safari support matters from the start; a future iPhone wrapper (e.g. Capacitor) can be evaluated after the gameplay is accepted. Prefer a clean, simple code structure, not an unnecessary multi-agent architecture or backend.

Use an actual game loop with delta-time / fixed-step collision simulation, separate graphics and physics coordinates, normalized responsive placement, and a single central set of gameplay configuration constants. No external servers or accounts needed.

4. Gameplay and controls

- Main menu -> Choose Difficulty (Easy, Medium, Hard) -> Play -> Pause/Resume -> Death/Results -> Retry or Menu. Provide a simple Info/How To Play view.
- Spawn ship in a safe, open-water location away from the cannon, shore and current cannonballs. Ship must move in ANY 2D direction, rotate visually to match heading, stay responsive and be constrained fully inside its smaller/inset ship boundary.
- **Desktop:** WASD and arrow keys, plus sensible mouse input where practical. **Mobile:** support landscape touch steering (responsive on-screen thumbstick or a clear touch-follow mechanic); do not depend on a physical keyboard. Prevent scrolling/touch zoom while playing.
- Fixed central island: cannon spins continuously; its current angle determines the outgoing ball direction. Randomize its start angle EVERY RUN for ALL difficulties. Optional slight angular jitter on each shot prevents trivial repeated patterns. Ball must spawn beyond the cannon muzzle/fort's collision area so it doesn't immediately collide with its origin.
- Ball velocity persists across ricochets; bounce off all coast segments, including the rounded-looking corners and bottom centre, with a specular reflection. Never allow a ball to disappear into or tunnel across cliffs or escape the arena after reflecting. Multiple balls remain alive until removed by the appropriate powerup or a new run starts.
- A cannonball touching the ship's actual collision hull results in immediate defeat, death explosion feedback, frozen final stats and a retry option. No lives system for this prototype.
- Central island is physically blocked for the ship. If the physics requires cannonball collisions against the island, implement them consistently without creating immediate self-collision; otherwise document intentional projectile pass-through clearly. The coastline is a shaped bounce surface, not simply an invisible 4-wall rectangle.
- Avoid random unfair unavoidable hits when spawning hazards. Never spawn coins or modifiers inside the island, too close to the cliffs, underneath the HUD, or on top of one another or the ship.
- No artificial cap on surviving cannonballs, except optional explicit performance safeguard that must not silently change challenge progression.

5. Difficulty configurations

These are the **latest intended numbers** from Versions 2–4. Put every figure in an editable `difficultyConfig` (exact tuning values can be adjusted after playtesting).

Parameter	Easy	Medium	Hard
Initial cannonball volley	1	3	5
First shot after start	~1 s	~1 s	~1 s
Additional volley shots	n/a	1–2 s apart	1–2 s apart
Starting ongoing fire interval	**5 s**	**4 s**	**3 s**
Relative cannon rotation speed	1.0x	1.5x	2.0x
Coin lifetime	4 s	3 s	2 s
Positive pickup lifetime	4 s	3 s	2 s
Negative pickup lifetime	4 s	5 s	6 s

Schedule the initial volley, then steady firing at each mode's interval. Do not accidentally schedule overlapping volley and steady shots. **All** modes gradually accelerate firing over time, with Medium accelerating more than Easy and Hard more than Medium. As an editable illustrative target, after a minute Easy is ~4 s rather than 5 s.

Set minimum intervals to avoid zero/negative durations. Starting angles vary per run, including Easy: do not launch repeatedly from the same angle across replays.

6. Collectibles and eight upgrades / downgrades

Coins spawn in safe random open-water positions, visibly bob/pulse, can be collected by the ship, disappear on pickup, and self-despawn according to difficulty. Separate the **coin count** from **score**. Use coin imagery in the game and in any 'Disable Coins' icon/text (the old gem mechanic was explicitly changed to coins). Powerups and downgrades should be clearly visually different: compass/pirate-themed tokens with **green cues for positive**, **red for negative**, recognizable icons, and very brief floating pickup messages. The table below is taken from the embedded Version 3 table, with the Version 4 capitalization/punctuation correction applied.

Type	Effect	Duration	On-pickup message
Positive	Make ship smaller (0.5x size; hitbox too)	10 s	**Blimey!**
Positive	Double score earned while active	10 s, configurable	**Booty!**
Positive	Remove ALL active cannonballs	Instant	**Parley!**
Positive	Speed up ship (suggest 1.5x)	10 s, configurable	**Yo-ho-ho!**
Negative	Make ship bigger (2x size; hitbox too)	10 s	**Landlubber!**
Negative	Speed up all cannonballs by 25%	10 s	**Shiver Me Timbers!**
Negative	Disable coins / despawn existing coins	10 s	**Plundered!**
Negative	Slow ship (suggest 0.5x movement speed)	10 s, configurable	**Weigh Anchor!**

Rules: timed effects expire correctly, restoring original values; retry clears all state and timers; picking up the same timed effect refreshes duration instead of compounding infinitely. Implement safe combinations (e.g. small + large ship, speed up + slow down) deterministically, and track actual cannonball speed when temporary speed changes start/end, including new balls fired during the effect. For disabled coins, prevent new spawns for 10 seconds. Every one of the four variants within a category should have an **equal default spawn chance**, but keep category frequencies separately configurable; Version 4 asked this question but did not document an answer. It should be tested for playability rather than assumed that upgrades, downgrades and coins all have identical total frequency.

7. Scoring, HUD and screens

Use a real score combining time survived AND coins, not simply the coin tally (a major V4 regression). Earlier notes gave an example formula rather than a mandatory final equation. For this prototype choose and document a configurable formula, e.g. `score = max(0, floor(10 * survivedSeconds) + 50 * coinsCollected + 20 * positivesCollected - 20 * negativesCollected)`, with the Double Score pickup doubling **points accrued during the active period** (not retroactively doubling lifetime score). The score should progress even if zero coins are collected. Make `score`, `time`, and `coins` consistently available on the play HUD and final results/death screen. Persist best local score per difficulty in `localStorage` (no remote leaderboard).

Show landscape play screen with a readable wood-style HUD anchored to the safe area at top, pause on right; don't overlap play controls with important objects or cut off UI on notched phones. Provide difficulty picker labels exactly **Easy**, **Medium**, **Hard**; no “Madium” typo. Main menu, difficulty selection, info, pause, and death/results need functioning navigation, not merely static mockups. The original polished menu/death assets are NOT included among these uploads; construct tasteful temporary UI matching the screenshot, and allow replacement later without rewriting game logic.

8. Audio, animation, feedback

Distinct sounds for cannon fire, collecting a coin, collecting a positive pickup, collecting a negative pickup (audible, not too quiet), and death/explosion; death must NOT reuse the cannon-fire sound. Add short cannon muzzle flash, coin pulse/bob, ship wake particles, small impact/explosion feedback, and quick text on pickup with correct sign/phrasing. Provide **separate music and sound-effect mute toggles** persisted locally. Gameplay uses upbeat pirate-style music; menu/death calmer pirate-style music. **No licensed music files were provided**, so use clearly documented placeholders, generated original audio if feasible, or a silent-but-working music toggle pending assets; never silently import commercial audio from the web. Audio should begin without avoidable multi-second delay, subject to browser/mobile user-gesture rules.

9. Borders, resolution and collision correctness

The source reports repeated bugs on mobile aspect ratios, ship clipping under cliffs, and cannonballs escaping (especially bottom middle). Treat these as acceptance criteria, not historical trivia.

- Match the **visible INNER waterline** of the illustrated irregular coast. Store an editable normalized path/ polygon or fine set of segments; curved or corner sections should behave as reflected boundaries, not easy straight rectangular cheats. Draw a temporary collision outline with a debug toggle to calibrate it to the sprite.
- Cannonball bounce contact centre is offset inward from the shore by cannonball radius; after each collision, resolve penetration, reflect relative to surface normal, and account for fast movement / multiple contacts in one time step. Use substeps, swept-circle tests, or equivalent robust collision response.
- Ship collision boundary is further **inset**, taking current ship size into account, so its visible artwork never vanishes under the coastline even at corners. Include central fort obstacle.
- All screen shapes and HUD positions must remain correct across at least desktop 16:9 and landscape iPhone proportions (e.g. 19.5:9). No accidental borders floating inside the viewport, black gaps from screen-size changes, or stretched collision geometry disagreeing with art. Prioritize visibility of all four sides, consistent gameplay distances and no unfair crop.
- Pause/resume freezes game timing, timers and motion correctly; resizing/rotating the screen doesn't teleport balls beyond bounds. Reject portrait gameplay gracefully with a Rotate Device prompt.

10. What to build NOW / what is out of scope

Deliver a playable end-to-end prototype, not just a project skeleton or written plan. Aim to implement the complete loop: title/difficulty -> responsive ship steering -> cannon -> bounces -> coins / eight modifiers -> death/results -> restart, with approximate original visuals and phone support. Where available, use supplied source assets. High-quality menus/audio may be provisional because their actual source files weren't supplied.

Defer: ads/IAP, shops/skins, downloadable maps, accounts, cloud saves, online leaderboards, Apple App Store packaging, multi-boat modes, bosses, level campaigns, elaborate monetization, or feature creep. The original idea mentioned future cosmetics and removing ads, but none are needed to validate this game.

11. Required engineering and acceptance tests

1. Repeated full sessions in each difficulty; correct 1/3/5 starting volleys and 5/4/3 base firing intervals, accelerated over time.
2. Start angle differs between new runs for ALL modes, especially Easy.
3. Cannonball bounces on all boundaries including irregular corners and **bottom-middle** without vanishing; no tunneling at max reasonable speed.
4. Ship stays fully visible within inset play area at default, half and double size; can never disappear behind cliff.
5. Distinct physics object vs artwork sizes, central island collision respected.
6. Coins and modifiers never spawn inside fort or off water; disappear on schedule by mode; coin-removed effect truly prevents spawning.
7. All **eight** pickup effects/messages/sfx work and properly reset/expire. No infinite cumulative speed changes.

8. Real-time score improves from survival alone; coins add points; score not equal to coin count; double-score works only while active; final screen shows results.
9. Desktop WASD/arrow keys and landscape touch input actually steer ship; pause and restart work.
10. UI scales to 16:9 and wide landscape mobile; no cropped HUD or wrong border/physics alignment.
11. Build succeeds via `npm run build`; include at least meaningful unit tests for bounce reflection, score and effect expiry; test via available browser runner/screenshots if possible.
12. A README details how to run, controls, asset mapping, tested limitations, and next steps. Capture at least one screenshot of gameplay to help visually compare with `original_gameplay_reference.jpg`.

12. Development workflow expected of Codex

Inspect repository first; don't overwrite existing unrelated files. Read this spec and asset guide. Implement in **logical milestones** and actually continue coding and testing, fixing errors until runnable. Configure scripts, provide any generated assets in the repo, and run available tests/build. Report succinctly: files changed, features working, screenshot(s), how to run, known gaps, and sensible next three improvements. Commit locally if normal for the environment; **do not push, deploy or publish without explicit instruction.** If something cannot be finished, say exactly what and provide a working subset; don't describe unfinished features as implemented.

Clarifications to keep visible

- Cannon location: **centre fort**, overriding the edge-of-arena concept in initial text.
- Latest difficulty base shot intervals: Easy **5s**, Medium **4s**, Hard **3s**, plus initial volleys and ramping speed.
- Pickup graphic: **coins**, not gems.
- Score: **time + coins + pickup adjustments**; earlier exact equation was merely illustrative.
- Prototype target: playable **browser first**, with touch responsiveness; native iPhone package later only if gameplay passes.
- Unprovided polished menu artwork, separated music and original sound tracks cannot be assumed to exist.